Enhancing Visually-Rich Document Understanding via Layout Structure Modeling

Qiwei Li
qw-line@whu.edu.cn
School of Computer Science, Wuhan University
Wuhan, Hubei, China

Zuchao Li*
zcli-charlie@whu.edu.cn
School of Computer Science, Wuhan University
Wuhan, Hubei, China

Xiantao Cai*
caixiantao@whu.edu.cn
School of Computer Science, Wuhan University
Wuhan, Hubei, China

Bo Du
dubo@whu.edu.cn
School of Computer Science, Wuhan University
Wuhan, Hubei, China

Hai Zhao
zhaohai@cs.sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

ABSTRACT

In recent years, the use of multi-modal pre-trained Transformers has led to significant advancements in visually-rich document understanding. However, existing models have mainly focused on features such as text and vision while neglecting the importance of layout relationship between text nodes. In this paper, we propose GraphLayoutLM, a novel document understanding model that leverages the modeling of layout structure graph to inject document layout knowledge into the model. GraphLayoutLM utilizes a graph reordering algorithm to adjust the text sequence based on the graph structure. Additionally, our model uses a layout-aware multi-head self-attention layer to learn document layout knowledge. The proposed model enables the understanding of the spatial arrangement of text elements, improving document comprehension. We evaluate our model on various benchmarks, including FUNSD, XFUND and CORD and it achieves state-of-the-art results among these datasets. Our experiment results demonstrate that our proposed method provides a significant improvement over existing approaches and showcases the importance of incorporating layout information into document understanding models. We also conduct an ablation study to investigate the contribution of each component of our model. The results show that both the graph reordering algorithm and the layout-aware multi-head self-attention layer play a crucial role in achieving the best performance.

KEYWORDS

Document Understanding; Information Extraction; Graph Structure; Layout Analysis

ACM Reference Format:
Qiwei Li, Zuchao Li*, Xiantao Cai*, Bo Du, and Hai Zhao. 2023. Enhancing Visually-Rich Document Understanding via Layout Structure Modeling. In *ACM Multimedia Asia Workshops (MMAsia '23 Workshops), December 06–08, 2023, Tainan, Taiwan.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3611380.3628554

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
MMAsia '23 Workshops, December 06–08, 2023, Tainan, Taiwan
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0326-3/23/12…$15.00
https://doi.org/10.1145/3611380.3628554

1 INTRODUCTION

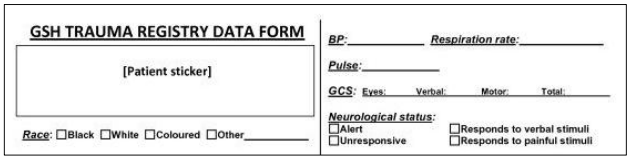
Visually-Rich Document Understanding (VRDU) [24] aims to analyze various types of scanned or digital-born documents that have a rich variety of structures and complex formats. It can assist in a wide range of document-related scenarios, such as report/receipt comprehension [19], document classification [8] and document question answering [18].

Unlike conventional natural language understanding (NLU), VRDU not only requires text information but also necessitates the integration of the original document structure and visual information to comprehend non-contiguous text positioned differently throughout the document [24, 27]. Document comprehension requires the combination of multi-modal information in a meaningful and efficient manner so that models can thoroughly explore the meaning and potential logical relationships within documents.

In the early stages of VRDU, researchers attempted uni-modal approaches and shallow multi-modal fusion approaches to analyze documents [2, 12, 21, 28]. These methods relied on pre-trained NLP and CV models separately and combined information from multiple modalities. However, such task-specific approaches required large amounts of annotated data for supervised learning. With recent advancements in pre-training techniques, many more sophisticated document models and pre-training approaches have been proposed. Examples include LayoutLM [24], LayoutLMv2 [27] and BROS [9], which have made significant progress in the VRDU field. The latest Document AI technique, LayoutLMv3 [10] and ERNIE-Layout [20], are new attempts at multi-modal pre-training and have achieved state-of-the-art results for many VRDU tasks.

Although current research has demonstrated superior performance in the VRDU field, there are still some limitations that need to be addressed.

(1) **Mismatch between the raw order and proper understanding order:** the text scattered in different location boxes of the document are non-continuous. This means that text from different location boxes needs to be joined together before model input. Some works [9, 10, 15, 24, 27] overlook the importance of reading



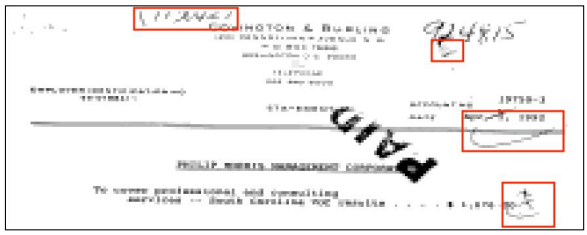
(a) Raw Order: *GSH TRAUMA REGISTERY DATA FORM BP: Respiration rate: [Patient sticker] Pulse: GCS: Eyes: Verbal: Motor: Total: Neurological status: Race: Black White Coloured Other Alert Responds to verbal stimuli Unresponsive Responds to painful stimuli*

(b) Proper Order: *GSH TRAUMA REGISTERY DATA FORM [Patient sticker] Race: Black White Coloured Other BP: Respiration rate: Pulse: GCS: Eyes: Verbal: Motor: Total: Neurological status: Alert Responds to verbal stimuli Unresponsive Responds to painful stimuli*

Figure 1: Different Reading Order. Raw Order shows fixed text reading order. Proper Order shows a reasonable and organized text reading order.



(a) Faulty Text Box



(b) Interference Information in Resized Image

Figure 2: Fig. (a) shows faulty text box (red box) in low quality document image. Fig. (b) shows interference information (red box) in the resized image.

order on the document understanding task and use an unadjusted raw order as the model input. As shown in Figure 1,it is difficult to acquire potential relationship between different text nodes by random or fixed text reading order. This can make it difficult to acquire potential relationships between different text nodes, which can impact machine understanding. A reasonable and organized reading order can reflect the logical or positional relationship between text nodes, making it more appropriate for disambiguation.

(2) **Inadequate disclosure of document structure by visual information:** some works [7, 9, 10, 15, 20, 24, 27] merely consider multi-modal vision and 2-dimensional (2D) position features to understand document layout . They only utilize multi-modal vision and 2D position features to understand document layout and engage the position and image features in the document understanding process. However, the model needs to learn the relationship between the text nodes only by itself, which can limit the model's understanding of the layout relationship. On one hand, low-quality document images can result in faulty text boxes and interference with image information. Thus, relying solely on vision features for document structure reflection is not sufficient. On the other hand, since the input size of the image encoder is limited, the usual practice is to resize the image to a fixed size that meets the requirements. As shown in Figure 2, during image preprocessing, LayoutLMv3 resizes the image to 224×224, which can lead to image blurriness and significant information loss, thus harming the layout understanding. Moreover, resizing further aggravates the damage caused by faulty text boxes and interference information since it is harder to distinguish them from normal information in a resized image. Additionally, the current use of 2D positions pre-extracted from the original image to indicate document structure is not intuitive. It is difficult for encoders to understand the structure of the document from 2D positions since the document structure is highly understood and related, rather than purely spatial.

In this work, we propose a VRDU pre-trained language model called GraphLayoutLM, which is an improved version of LayoutLMv3 and utilizes a layout relationship graph to improve document understanding. Unlike previous models that focus on providing feature information, GraphLayoutLM concentrates on the position relationship between text nodes and represents this kind of layout relationship using a graph structure. To build GraphLayoutLM, we introduce two optimization strategies: graph sequence optimization and graph mask optimization.

First, we model the document layout information as a hierarchical relational structure graph consisting of multiple text nodes. For example, a paragraph comprises several lines of text and paragraphs form sections that together make up the entire document. This hierarchical document structure serves as the foundation for constructing the document layout graph. To describe the hierarchical document structure, we pre-build the document layout as a tree. Since siblings usually share a closer relationship than non-siblings, we establish a position relationship between sibling nodes to enrich the tree into a layout graph.

The order of reading documents can vary widely and a proper sequencing can help us better comprehend the content. To achieve a suitable reading order, we utilize the layout graph built on a tree-based structure that preserves the original hierarchical information. Our approach involves a deep traversal of the layout tree, which arranges closely related nodes adjacent to their relevant position in the sequence. We also sort sibling nodes based on their position information in the graph. As our graph sequence optimization approach, reordering by graph enables the model to obtain an optimal reading order, which serves as an augmentation strategy to enhance document understanding performance.

Inspired by Graph Attention Networks (GAT) [22], we attempt to integrate layout graph information into the VRDU model through a self-attention strategy. GraphLayoutLM introduces a graph-aware self-attention layer that enhances the standard self-attention layers by optimizing the graph mask. It is achieved by incorporating layout graph information through an adjacent matrix. The graph mask selectively masks attention score elements that lack edge

relationships between text nodes and inject the relationship between text nodes. This approach enables the model to better focus on the relationship between correlated text nodes, enhancing its performance.

We conduct extensive experiments on typical VRDU tasks to evaluate our proposed GraphLayoutLM model, including three datasets: the FUNSD and XFUND dataset for form understanding, and the CORD dataset for receipt understanding. The experiments show that GraphLayoutLM achieves a new state-of-the-art (SOTA) performance on these three datasets, significantly outperforming the previous SOTA baseline LayoutLMv3 by 2.29% and 1.54% on FUNSD and XFUND, respectively. Additionally, we perform ablation studies and the results show the effectiveness of our proposed graph reordering and graph masking strategies. Our code will be available at https://github.com/Line-Kite/GraphLayoutLM.

2 GRAPHLAYOUTLM

We propose a novel GraphLayoutLM, which is based on the currently strongest model, LayoutLMv3. An overview of our proposed GraphLayoutLM can be seen in Figure 3.

2.1 Basis Architecture

2.1.1 Multi-modal Embeddings. Consistent with LayoutLMv3, we converted text, position and image inputs into text embeddings and image embeddings before being fed into the multi-modal Transformer. The text embedding is composed of the text itself, along with 1D and 2D positional information. The 1D position refers to the index of the text in the sequence, while the 2D layout position represents the border coordinates of the text. After the embedding process, these three components are added together to form the text embedding. Meanwhile, the image embedding follows the same approach as ViT [6] for processing images. In LayoutLMv3, the document image is first adjusted to a size of $H \times W$. If the image is represented by $I \in \mathbb{R}^{C \times H \times W}$, where C, H and W represent the channel size, width and height, respectively, the image is divided into uniform sequences of $P \times P$ blocks. These image blocks are linearly projected into vectors of dimension D and flattened into vector sequences of length $M = HW/P^2$. Finally, the text embedding and image embedding are concatenated and input into the following multi-modal Transformer encoder.

2.1.2 Multi-modal Transformer. In line with the LayoutLMv3 approach, we make some adjustments to the multi-modal Transformer in GraphLayoutLM.

Firstly, we use $\mathbf{b}^{(1D)}$, $\mathbf{b}^{(2D_x)}$ and $\mathbf{b}^{(2D_y)}$ to represent the learnable 1D and 2D position biases which vary across attention heads but are shared in all encoder layers. Using (x_i, y_i) as the anchor for the top-left corner of the i-th bounding box, we can compute the spatial-aware attention score:

$$a'_{i,j} = a_{i,j} + \mathbf{b}^{(1D)}_{j-i} + \mathbf{b}^{(2D_x)}_{x_j - x_i} + \mathbf{b}^{(2D_y)}_{y_j - y_i}, \tag{1}$$

where a_{ij} is the original attention score. Moreover, we follow CogView [5] and modify the attention calculation formula for self-attention:

$$\text{Softmax}\left(\frac{\mathbf{Q}^T\mathbf{K}}{\sqrt{d}}\right) = \text{Softmax}\left(\left(\frac{\mathbf{Q}^T}{\alpha\sqrt{d}}\mathbf{K} - \max\left(\frac{\mathbf{Q}^T}{\alpha\sqrt{d}}\mathbf{K}\right)\right) \times \alpha\right), \tag{2}$$

where α is scaling factor.

2.1.3 Pre-training Objectives. Our primary goal is to enhance VRDU performance through layout graphs, we still adopt three types of unsupervised pre-training methods: MLM, MIM and WPA, similar to those used in LayoutLMv3. By incorporating these techniques, our complete pre-training objective is defined as $L = L_{MLM} + L_{MIM} + L_{WPA}$. The detailed introduction of pre-training objectives is shown in Appendix A.1.

2.2 Layout Graph Modeling

In GraphLayoutLM, we treat each basic text node as a segment-level entity, representing a line of text. The entire document is comprised of these text lines, which can be described as a set of nodes $N = \{n_1, ..., n_m\}$. However, not all text nodes in these lines are correlated. To find associations between texts and avoid redundant connections, we extract document hierarchy information such as chapters and paragraphs using OCR tools or layout information from the data. For instance, consider a paragraph that contains text lines which can be defined as $P = \{n_1, ..., n_k\}$, where P is the paragraph region. We determine the text node representing the whole paragraph, n_p, based on its 2D position:

$$L(n_i) = \begin{cases} 1, & box_i = Top(P) \quad and \quad box_i = Left(P) \\ 0, & otherwise \end{cases}, \tag{3}$$

where the function $L(n_i)$ determines whether or not n_i can represent the entire paragraph, i.e., $n_p = n_i$. This determination is based on 2D position information of n_i, denoted as box_i, as well as the 2D position information of the top-most node and left-most node in P, represented by $Top(P)$ and $Left(P)$, respectively. This function is designed to mimic the way in which people typically write documents.

After determining the representation text node n_p of the paragraph P, we connect n_p with other nodes in the paragraph except for n_p with "*Parent-Child*" relationship to construct a subtree T_p, which is formulated as follows:

$$T_p =< n_p, \complement_P n_p, \textit{Parent-Child} >, \tag{4}$$

where $\complement_P n_p$ means the complement set of n_p in P.

Using the method described above, we can obtain a set of subtrees from the document, denoted as $T = \{T_1, ..., T_n\}$. However, to obtain complete trees instead of forests, we need to introduce a pseudo-global root node, n_g, which connects all the root nodes of the subtrees. By doing this, we can create a document layout tree, T_g.

The layout tree reflects the hierarchical structure of the document, with parent nodes and child nodes. However, the relative spatial relationships between nodes within a level are also an important aspect of the document's structural information and are useful for understanding the document. To fully utilize the relative spatial information of text nodes, we compare the positions of sibling nodes in the document tree T_g to determine their spatial relationship.

Figure 4 shows the process of establishing the position relationship of sibling nodes. Suppose that the 2D position of a node n_i is $n_i.box = [x_1^i, y_1^i, x_2^i, y_2^i]$, where (x_1^i, y_1^i) and (x_2^i, y_2^i) represents the coordinates of the two points in the lower left and upper right

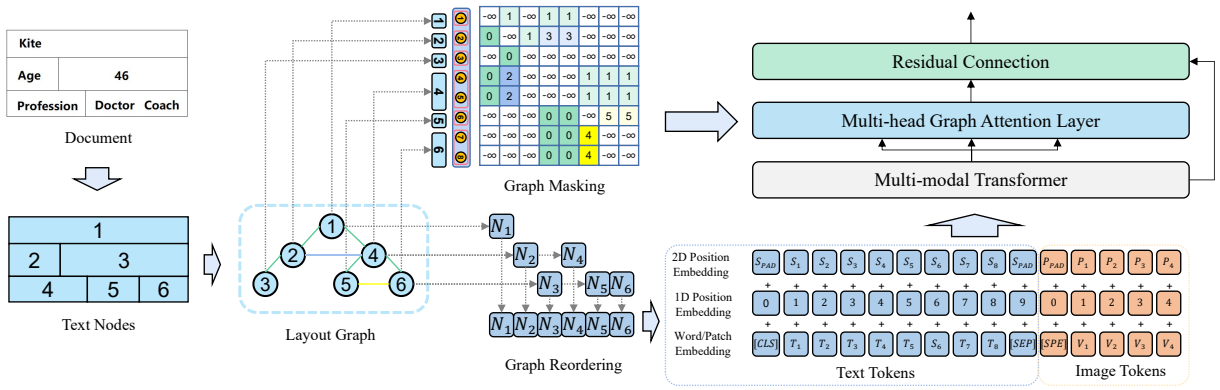


Figure 3: The architecture of GraphLayoutLM. In the text extraction stage, the document hierarchy needs to be extracted to generate the document layout graph. The reading order of text tokens sequence is adjusted by Graph Reordering before entering the Multi-modal Transformer. In the output part of Multi-modal Transformer, we use residual connection to connect a multi-head self-attention layer, so as to add the graph mask information for relationship learning.

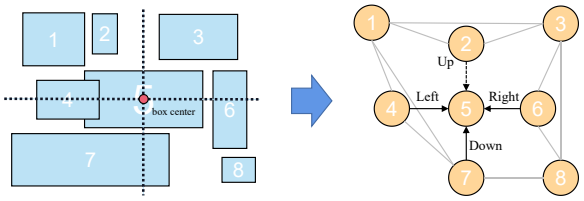


Figure 4: An example to establish the position relationship of sibling nodes.

corners of the rectangular region in which node n_i is located. We take the center point of $n_i.box$ of node n_i as the origin, and draw horizontal judgment line $l_x^i = (x_1^i + x_2^i)/2$ and vertical judgment line $l_y^i = (y_1^i + y_2^i)/2$. In the comparison process, if the 2D position $n_j.box$ of node n_j intersects l_x^j or l_y^j, the relationship will be established according to its 2D position, which can be expressed as:

$$rel(n_i, n_j) = \begin{cases} Left, & l_x^i \text{ intersects } n_j.box \text{ on the left side} \\ Right, & l_x^i \text{ intersects } n_j.box \text{ on the right side} \\ Up, & l_y^i \text{ intersects } n_j.box \text{ at the top} \\ Down, & l_y^i \text{ intersects } n_j.box \text{ at the bottom} \\ None, & otherwise \end{cases} . \quad (5)$$

Note that if the relationship between two nodes is unidirectional, we can make it bidirectional by adding the inverse relationship that is based on the original unidirectional relationship.

Completed by positional relationships, the relationship tree T_g will be enriched into the document graph structure G:

$$G = T_g \cup \{< n_i, n_j, rel(n_i, n_j) > | n_i, n_j \in T_g.sibling\}, \quad (6)$$

where $T_g.sibling$ means the set of sibling nodes in T_g.

2.3 Graph Reordering Strategy

By modeling the layout as a graph, we can optimize the input sequence of text to facilitate document understanding. However, some existing approaches, such as XYLayoutLM [7], focus solely on the relative position in the 2D position layout, ignoring the hierarchy that can be captured from visual information.

Our layout graph is designed to capture comprehensive spatial relationships, which allows for greater flexibility in optimizing reading sequences. This is especially important because different people read documents in different orders, so a reasonable reading order may vary. Additionally, we believe that the reading order extracted from our document layout graph can help the model identify potential associations between text nodes, improving overall comprehension. We show a reordering example in Appendix A.3. The overall graph reordering strategy is shown in Algorithm 1.

Hierarchical Reordering. As discussed in subsection 2.2, the document relationship graph consists of a layout document tree T_g, which represents the hierarchical structure of the document layout. The hierarchical reordering involves a depth-first traversal of the entire relational structure tree starting from the root node. It is important to note that the root node holds no meaningful information and should be removed from the sequence during input sequence construction. During the traversal, nodes are arranged in order of the parent, child and finally sibling nodes. This depth-first traversal ordering strengthens connections between nodes and their adjacent regions, as sibling nodes are always located within the same region.

Sibling Reordering. Hierarchical reordering optimizes the input sequence of text based on its hierarchical relationship. However, to properly order sibling nodes that share the same parent, we need an additional rule to adjust their sequence order at the same level. For this purpose, we use box position information of text nodes to rank the text sequence at the same level according to the position relationship between text nodes. The logic behind position reordering is to sort sibling nodes from top-to-bottom and left-to-right. Specifically, text nodes located at the top of the document are preferred. If the vertical position is roughly the same, then horizontal position is compared, with the left side taking preference. This reordering method is more in line with human habits of writing and reading documents, enabling the model to

better understand the logical relationship between text nodes at the same level.

Algorithm 1 Graph Reorder Algorithm

Require: G
Ensure: $order = Reorder(G)$
  $order \leftarrow empty$
  $stack \leftarrow empty$
  $stack.push(G.root)$
  while $stack$ is not empty **do**
    $node \leftarrow stack.pop()$
    $order.put(node)$
    $sort \leftarrow empty$
    while $node.children$ is not empty **do**
      $child \leftarrow node.children.pop()$
      $index \leftarrow 0$
      while index is not out of the range of $sort$ **do**
        $temp \leftarrow sort[index]$
        if $temp$ at the top or left of $child$ **then**
          $index \leftarrow index + 1$
        else
          break
        end if
      end while
      $sort.insert(index, child)$
    end while
    while $sort$ is not empty **do**
      $child \leftarrow sort.pop()$
      $stack.push(child)$
    end while
  end while

2.4 Graph Masking Strategy

The document layout graph contains rich and clear information about the document structure. In addition to optimizing the order of text input, it can improve document representation learning. The graph presents the position and hierarchical relationships among its text nodes in the form of edges, which can reveal logical connections between nodes. For instance, in a typical Q&A format, the answer text node typically appears on the right or lower side of the question text node. To incorporate this graph information into the document representation learning, we propose a novel graph masking strategy.

Figure 3 illustrates the surjective mapping between text tokens ($T = \{t_1, ..., t_l\}$) and text nodes ($N = \{n_1, ..., n_m\}$). To implement graph masking in Transformer encoding, we convert the relationships between nodes in a graph to relationships between text tokens. Specifically, any text token $t_i \in n_a$ in node n_a and any text token $t_j \in n_b$ in node n_b have the same relationship $rel(t_i, t_j) = rel(n_a, n_b)$ as their corresponding nodes. This completes the mapping from relationships between nodes to relationships between text tokens.

Although the document is recognized by OCR and linearized into a language sequence, its sequence differs from that of natural language. Tokens placed in close proximity may not necessarily have strong correlations; they are likely arranged that way due to linearization. Conversely, tokens that are far apart may not have smaller relationships; they may only appear distant because there are many text nodes at the same level. Therefore, to directly reflect the document's structure from an encoding perspective, we use a graph masking strategy. Specifically, we propose an improved Graph-aware Transformer layer that calculates the self-attention score of the text representation and adds a graph mask M_g, containing relational information, in the calculation process. This drives the model to pay more attention to relevant parts of the layout and ignore irrelevant ones, thereby reducing the impact of irrelevant information on understanding. The model becomes aware of document structures as a result. For the graph mask $M \in \mathbb{R}^{n \times n}$, element $m_{i,j}$ of M is computed by:

$$m_{i,j} = \begin{cases} w_{edge}(t_i, t_j), & edge(t_i, t_j) \in G \\ -9e15, & edge(t_i, t_j) \notin G \end{cases}, \tag{7}$$

where $w_{edge}(t_i, t_j)$ is the weight of relation labels between tokens t_i and t_j.

In our graph-aware Transformer, it also employs multi-head self-attention for text encoding, in which the self-attention score $e_{i,j}$ between two tokens t_i and t_j can be formulated as:

$$e_{i,j} = \frac{\mathbf{W}_Q h_i (\mathbf{W}_K h_j)^T}{\sqrt{d_k}} + m_{i,j}, \tag{8}$$

where $\mathbf{W}_Q$ and $\mathbf{W}_K$ are learnable projection weights for self-attention, h_i and h_j are token representation input for tokens t_i and t_j respectively.

Because of the existence of graph masking, the model will selectively ignore the elements without edge relationship in the process of self-attention learning, and then inject the layout structure into text representations. The final text representation H_g calculated by graph-aware self-attention with layout is as follows:

$$H_g = \text{Linear}(\text{Softmax}(\frac{\mathbf{W}_Q H (\mathbf{W}_K H)^T}{\sqrt{d_k}} + M_g) \cdot \mathbf{W}_V H), \tag{9}$$

where H is the input text representations, W_V is also the learnable value projection weight.

To prevent potential information loss caused by masking resulting from layout graph construction errors, we use residual connections to merge the input text representation with the output representation that is aware of the layout graph. Residual connections are useful because they allow us to retain information from the input representation and combine it with the output representation in a way that improves model performance. Ultimately, this combination results in a final output that can be used for downstream tasks:

$$H_{output} = \textsc{Residual}(H, H_g). \tag{10}$$

3 EXPERIMENTS

3.1 Main Analysis

The results for FUNSD and CORD are presented in Table 1. For both base and large model sizes, GraphLayoutLM demonstrates improved precision, recall and F1 scores compared with the baseline model, LayoutLMv3. On the FUNSD dataset, it can be observed that GraphLayoutLM achieved an F1 score of 93.15 with the base size, which is 1.96 higher than the original LayoutLMv3$_{\text{BASE}}$ F1 score of 91.19. Moreover, the base size GraphLayoutLM F1 score

Table 1: Results on FUNSD and CORD test sets. "T/L/I/G" denotes "text/layout/image/graph" modality. Results for LayoutLMv3 that are labeled with "†" indicate that the results are obtained through replication in our experiments.

Model	Parameters	Modality	FUNSD			CORD		
			Precision	Recall	F1	Precision	Recall	F1
$BERT_{BASE}$ [4]	110M	T	54.69	67.10	60.26	88.33	91.07	89.68
$RoBERTa_{BASE}$ [17]	125M	T	63.49	69.75	66.48	-	-	93.54
$BROS_{BASE}$ [9]	110M	T+L	81.16	85.02	83.05	95.58	95.14	95.36
$LayoutLM_{BASE}$ [24]	160M	T+L+I	76.77	81.95	79.27	94.37	95.08	94.72
$XYLayoutLM_{BASE}$ [7]	-	T+L+I	-	-	83.35	-	-	-
$LayoutLMv2_{BASE}$ [27]	200M	T+L+I	80.29	85.37	82.76	94.53	95.39	94.95
$DocFormer_{BASE}$ [1]	183M	T+L+I	80.76	86.09	83.34	96.52	96.14	96.33
$ERNIE\text{-}Layout_{BASE}$ [20]	-	T+L+I	-	-	90.28	-	-	96.61
$LayoutLMv3_{BASE}$ [10]	133M	T+L+I	-	-	90.29	-	-	96.56
$LayoutLMv3_{BASE}$ [10]†	133M	T+L+I	90.82	91.55	91.19	96.35	96.71	96.53
$GraphLayoutLM_{BASE}$ (Ours)	135M	T+L+I+G	**92.46**	**93.85**	**93.15**	**97.02**	**97.53**	**97.28**
$BERT_{LARGE}$ [4]	340M	T	61.13	70.85	65.63	88.86	91.68	90.25
$RoBERTa_{LARGE}$ [17]	355M	T	67.80	73.91	70.72	-	-	93.80
$LayoutLM_{LARGE}$ [24]	343M	T+L	75.96	82.19	78.95	94.32	95.54	94.93
$BROS_{LARGE}$ [9]	340M	T+L	82.81	86.31	84.52	-	-	97.28
$StructuralLM_{LARGE}$ [15]	355M	T+L	83.52	86.81	85.14	-	-	-
$LayoutLMv2_{LARGE}$ [27]	426M	T+L+I	83.24	85.19	84.20	95.65	96.37	96.01
$DocFormer_{LARGE}$ [1]	536M	T+L+I	82.29	86.94	84.55	97.25	96.74	96.99
$ERNIE\text{-}Layout_{LARGE}$ [20]	-	T+L+I	-	-	93.12	-	-	97.21
$LayoutLMv3_{LARGE}$ [10]	368M	T+L+I	-	-	92.08	-	-	97.46
$LayoutLMv3_{LARGE}$ [10]†	368M	T+L+I	91.51	92.70	92.10	97.45	97.52	97.49
$GraphLayoutLM_{LARGE}$ (Ours)	372M	T+L+I+G	**94.49**	**94.30**	**94.39**	**97.75**	**97.75**	**97.75**

is also 1.05 higher than the $LayoutLMv3_{LARGE}$ F1 score of 92.10, and even surpasses the previous SOTA, $ERNIE\text{-}Layout_{LARGE}$ F1 score of 93.12. The large GraphLayoutLM F1 score (94.39) is higher than the previous SOTA (93.12). These results serve as evidence of our GraphLayoutLM's exceptional performance and its significant improvement over LayoutLMv3.

On the CORD dataset, the $GraphLayoutLM_{BASE}$ F1 score is 97.28, which is 0.75 higher than the $LayoutLMv3_{BASE}$ F1 score of 96.53. The $GraphLayoutLM_{LARGE}$ F1 score is 97.75, which is 0.26 higher than the $LayoutLMv3_{LARGE}$ F1 score of 97.49. It can be observed that GraphLayoutLM has less effect on promotion for CORD. This may be because CORD is a receipt dataset, where document text content is minimal and the format is simple. Layout graphs are more suitable for aiding models understand complex documents. However, even in this scenario, GraphLayoutLM still provides extra enhancement and has become the new SOTA for the CORD dataset.

In this table, we have also included the number of parameters for each model. It is evident that the GraphLayoutLM base and large size models only have 2M and 4M more parameters than their corresponding LayoutLMv3 models, respectively. This further emphasizes that our strategies can yield significant performance enhancements with just a few additional parameters.

To assess the language-independence of our GraphLayoutLM model, we present the results on the Chinese dataset, XFUND, in Table 2. As only F1 scores are publicly available for the baseline models on XFUND, we compare only the F1 scores in the experiment. Based on comparative experiments, we find that GraphLayoutLM achieves SOTA with an F1 score of 93.56, which is higher than the original SOTA LayoutLMv3 F1 score of 92.02. These experiment findings demonstrate that GraphLayoutLM showcases good adaptability and delivers excellent performance across multiple languages.

Table 2: Results on XFUND. The LayoutLMv3 labeled with "†" means the experiment results that we reproduce. The models labeled with "‡" mean that we get the results from github.

Model	Modality	XFUND F1
$XLM\text{-}RoBERTa_{BASE}$ [3]	T	87.74
$XLM\text{-}RoBERTa_{LARGE}$ [3]	T	89.25
$LayoutXLM_{BASE}$ [25]	T+L+I	89.24
$LayoutXLM_{LARGE}$ [25]	T+L+I	91.61
$XYLayoutLM_{BASE}$ [7]	T+L+I	91.76
$ERNIE\text{-}LayoutX_{BASE}$ [20]‡	T+L+I	88.58
$LayoutLMv3\text{-}Chinese_{BASE}$ [10]‡	T+L+I	92.02
$LayoutLMv3\text{-}Chinese_{BASE}$ [10]†	T+L+I	91.82
$GraphLayoutLM\text{-}Chinese_{BASE}$(Ours)	T+L+I+G	**93.56**

4 CONCLUSION

In this paper, we propose GraphLayoutLM - a layout graph-based multi-modal model for document understanding. We focus on the layout structure of documents and design a novel layout graph construction method. Based on the layout graph, we propose graph reordering and graph masking strategies. Our experiments show that GraphLayoutLM exhibits superior performance and achieves new state-of-the-art results in the FUNSD, CORD and XFUND datasets, shedding light on the document understanding task.

REFERENCES

[1] Srikar Appalaraju, Bhavan Jasani, Bhargava Urala Kota, Yusheng Xie, and R Manmatha. 2021. Docformer: End-to-end transformer for document understanding. In *Proceedings of the IEEE/CVF international conference on computer vision*. 993–1003.

[2] Mengli Cheng, Minghui Qiu, Xing Shi, Jun Huang, and Wei Lin. 2020. One-shot text field labeling using attention and belief propagation for structure information extraction. In *Proceedings of the 28th ACM International Conference on Multimedia*. 340–348.

[3] Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Unsupervised cross-lingual representation learning at scale. *arXiv preprint arXiv:1911.02116* (2019).

[4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).

[5] M. Ding, Z. Yang, W. Hong, W. Zheng, C. Zhou, D. Yin, J. Lin, X. Zou, Z. Shao, and H. Yang. 2021. CogView: Mastering Text-to-Image Generation via Transformers. (2021).

[6] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. 2020. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929* (2020).

[7] Zhangxuan Gu, Changhua Meng, Ke Wang, Jun Lan, Weiqiang Wang, Ming Gu, and Liqing Zhang. 2022. Xylayoutlm: Towards layout-aware multimodal networks for visually-rich document understanding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 4583–4592.

[8] Adam W Harley, Alex Ufkes, and Konstantinos G Derpanis. 2015. Evaluation of deep convolutional nets for document image classification and retrieval. In *2015 13th International Conference on Document Analysis and Recognition (ICDAR)*. IEEE, 991–995.

[9] Teakgyu Hong, Donghyun Kim, Mingi Ji, Wonseok Hwang, Daehyun Nam, and Sungrae Park. 2022. Bros: A pre-trained language model focusing on text and layout for better key information extraction from documents. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36. 10767–10775.

[10] Yupan Huang, Tengchao Lv, Lei Cui, Yutong Lu, and Furu Wei. 2022. Layoutlmv3: Pre-training for document ai with unified text and image masking. In *Proceedings of the 30th ACM International Conference on Multimedia*. 4083–4091.

[11] Guillaume Jaume, Hazim Kemal Ekenel, and Jean-Philippe Thiran. 2019. Funsd: A dataset for form understanding in noisy scanned documents. In *2019 International Conference on Document Analysis and Recognition Workshops (ICDARW)*, Vol. 2. IEEE, 1–6.

[12] Anoop Raveendra Katti, Christian Reisswig, Cordula Guder, Sebastian Brarda, Steffen Bickel, Johannes Höhne, and Jean Baptiste Faddoul. 2018. Chargrid: Towards understanding 2d documents. *arXiv preprint arXiv:1809.08799* (2018).

[13] Chen-Yu Lee, Chun-Liang Li, Timothy Dozat, Vincent Perot, Guolong Su, Nan Hua, Joshua Ainslie, Renshen Wang, Yasuhisa Fujii, and Tomas Pfister. 2022. Formnet: Structural encoding beyond sequential modeling in form document information extraction. *arXiv preprint arXiv:2203.08411* (2022).

[14] Chen-Yu Lee, Chun-Liang Li, Chu Wang, Renshen Wang, Yasuhisa Fujii, Siyang Qin, Ashok Popat, and Tomas Pfister. 2021. Rope: reading order equivariant positional encoding for graph-based document information extraction. *arXiv preprint arXiv:2106.10786* (2021).

[15] Chenliang Li, Bin Bi, Ming Yan, Wei Wang, Songfang Huang, Fei Huang, and Luo Si. 2021. Structurallm: Structural pre-training for form understanding. *arXiv preprint arXiv:2105.11210* (2021).

[16] Xiaojing Liu, Feiyu Gao, Qiong Zhang, and Huasha Zhao. 2019. Graph convolution for multimodal information extraction from visually rich documents. *arXiv preprint arXiv:1903.11279* (2019).

[17] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692* (2019).

[18] Minesh Mathew, Dimosthenis Karatzas, and CV Jawahar. 2021. Docvqa: A dataset for vqa on document images. In *Proceedings of the IEEE/CVF winter conference on applications of computer vision*. 2200–2209.

[19] Seunghyun Park, Seung Shin, Bado Lee, Junyeop Lee, Jaeheung Surh, Minjoon Seo, and Hwalsuk Lee. 2019. CORD: a consolidated receipt dataset for post-OCR parsing. In *Workshop on Document Intelligence at NeurIPS 2019*.

[20] Qiming Peng, Yinxu Pan, Wenjin Wang, Bin Luo, Zhenyu Zhang, Zhengjie Huang, Teng Hu, Weichong Yin, Yongfeng Chen, Yin Zhang, et al. 2022. ERNIE-Layout: Layout Knowledge Enhanced Pre-training for Visually-rich Document Understanding. *arXiv preprint arXiv:2210.06155* (2022).

[21] Ritesh Sarkhel and Arnab Nandi. 2019. Deterministic routing between layout abstractions for multi-scale classification of visually rich documents. In *28th International Joint Conference on Artificial Intelligence (IJCAI), 2019*.

[22] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. *arXiv preprint arXiv:1710.10903* (2017).

[23] Wenjin Wang, Zhengjie Huang, Bin Luo, Qianglong Chen, Qiming Peng, Yinxu Pan, Weichong Yin, Shikun Feng, Yu Sun, Dianhai Yu, et al. 2022. ERNIE-mmLayout: Multi-grained MultiModal Transformer for Document Understanding. *arXiv preprint arXiv:2209.08569* (2022).

[24] Yiheng Xu, Minghao Li, Lei Cui, Shaohan Huang, Furu Wei, and Ming Zhou. 2020. Layoutlm: Pre-training of text and layout for document image understanding. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 1192–1200.

[25] Yiheng Xu, Tengchao Lv, Lei Cui, Guoxin Wang, Yijuan Lu, Dinei Florencio, Cha Zhang, and Furu Wei. 2021. Layoutxlm: Multimodal pre-training for multilingual visually-rich document understanding. *arXiv preprint arXiv:2104.08836* (2021).

[26] Yiheng Xu, Tengchao Lv, Lei Cui, Guoxin Wang, Yijuan Lu, Dinei Florencio, Cha Zhang, and Furu Wei. 2022. XFUND: A Benchmark Dataset for Multilingual Visually Rich Form Understanding. In *Findings of the Association for Computational Linguistics: ACL 2022*. 3214–3224.

[27] Yang Xu, Yiheng Xu, Tengchao Lv, Lei Cui, Furu Wei, Guoxin Wang, Yijuan Lu, Dinei Florencio, Cha Zhang, Wanxiang Che, et al. 2020. Layoutlmv2: Multimodal pre-training for visually-rich document understanding. *arXiv preprint arXiv:2012.14740* (2020).

[28] Zichao Yang, Diyi Yang, Chris Dyer, Xiaodong He, Alex Smola, and Eduard Hovy. 2016. Hierarchical attention networks for document classification. In *Proceedings of the 2016 conference of the North American chapter of the association for computational linguistics: human language technologies*. 1480–1489.

A APPENDICES

A.1 Pre-training Objectives

Masked Language Modeling (MLM)

To make use of the text input, we employ an unsupervised pre-training target called MLM, which is similar to BERT. We mask 30% of the text tokens and aim to maximize the log-likelihood of the correct masked text tokens (represented as $\mathbf{y}_l$) as the pre-training objective. The text tokens are derived from corrupted sequences of image tokens ($\mathbf{X}^{I_m}$) and text tokens ($\mathbf{Y}^{L_m}$) based on their contextual representations, where I_m and L_m represent the masked positions. The MLM loss can be represented as follows:

$$L_{MLM}(\theta) = -\sum_{l=1}^{|L_m|} \log P_\theta \left(\mathbf{y}_\ell \mid \mathbf{X}^{I_m}, \mathbf{Y}^{L_m} \right), \tag{11}$$

where θ denotes the parameters of our GraphLayoutLM model.

Masked Image Modeling (MIM)

As in the patch-level MIM pre-trained ViT model [6], we use a blockwise masking strategy to randomly mask approximately 40% of the image tokens. The MIM objective is also driven by cross-entropy loss, similar to MLM objective, to reconstruct the masked image tokens x_m based on the context of their surrounding text and image tokens:

$$L_{MIM}(\theta) = -\sum_{m=1}^{|I_m|} \log P_\theta \left(\mathbf{x}_m \mid \mathbf{X}^{I_m}, \mathbf{Y}^{L_m} \right). \tag{12}$$

Word-Patch Alignment (WPA)

The LayoutLMv3 paper proposes a WPA objective for learning fine-grained alignment between text words and image patches, which is useful for learning alignment between text and image. We follow this objective and use an additional two-layer Multi-Layer Perceptron (MLP) network that inputs contextual text and images and predicts binary aligned/misaligned labels. The WPA objective is implemented by a binary cross-entropy loss:

$$L_{WPA}(\theta) = -\sum_{\ell=1}^{L-L_m} \log P_\theta \left(\mathbf{z}_\ell \mid \mathbf{X}^{I_m}, \mathbf{Y}^{L_m} \right), \tag{13}$$

Table 3: Finetuning hyper-parameters. L, M, B and G refer to learning rate, max steps, batch size and gradient accumulation steps.

Dataset	Model size	Language	L	M	B	G
FUNSD	BASE	English	1e-5	2000	4	1
	LARGE		1e-5	2000	4	1
CORD	BASE	English	5e-5	4000	4	1
	LARGE		1e-5	8000	4	1
XFUND	BASE	CHINESE	7e-5	2000	4	4

where $L - L_m$ is the number of unmasked text tokens, $\mathbf{z}_\ell$ is the binary label of language token in the ℓ position.

A.2 Experimental Setup

Dataset. We conduct form and receipt understanding experiments on several widely-used VRDU datasets, including FUNSD [11] and CORD [19] following LayoutLMv3. FUNSD is a form understanding dataset for scanned documents that contains 199 annotated documents (149 in the training set and 50 in the test set) with a total of 9,707 semantic entity labels. Our focus is on the semantic entity labeling task, where each entity is assigned one of four labels: "question," "answer," "header," or "other." Similarly, CORD is a receipt understanding dataset consisting of 1,000 receipts split into 800 training, 100 validation and 100 test examples. The dataset defines 30 semantic labels under four broad categories for the receipt key information extraction task.

Furthermore, we conduct additional experiments on the XFUND [26] dataset, which is a manually annotated multilingual document understanding dataset containing 199 documents in seven languages. While XFUND also includes annotations in multiple languages, we focus specifically on the Chinese versions due to pre-training limitations. For the Chinese version of XFUND, it includes 13,857 semantic entity annotations. Like FUNSD, this dataset is divided into a 149-sample training set and a 50-sample test set, and our task is to label each semantic entity as "question," "answer," "header," or "other."

Model Details. To ensure a fair comparison, GraphLayoutLM's network architecture follows that of LayoutLMv3. The GraphLayoutLM$_{\text{BASE}}$ model employs a 12-layer Transformer encoder with 12-head self-attention, a hidden size of $D = 768$ and 3,072 intermediate dimensions for feed-forward networks. The number of heads in the graph-aware multi-head self-attention layer is 6. In contrast, the GraphLayoutLM$_{\text{LARGE}}$ model uses a 24-layer Transformer encoder with 16-head self-attention, a hidden size of $D = 1,024$ and 4,096 intermediate dimensions for feed-forward networks. The number of heads in the graph-aware multi-head self-attention layer is 8. For both models, we set the maximum text sequence length to $L = 512$ and the image input size to $C \times H \times W = 3 \times 224 \times 224$. The image patch size P is 16 and the visual sequence length M is 196. We also set the Transformer self-attention layer scaling factor α to 32.

To ensure a fair comparison with LayoutLMv3, we adopt similar settings for our model's pre-training, including the data used and the pre-training steps. For fine-tuning on downstream tasks, we set hyper-parameters consistent with those used in LayoutLMv3. The detailed hyper-parameters are shown in Table 3.

. These measures help ensure that any differences in performance between the models can be attributed to architectural changes rather than variations in training procedures.

Baselines. We compare our model against several baselines, including typical self-supervised pre-training language models such as BERT [4] and RoBERTa [17], as well as document understanding models that focus on text and layout modalities like BROS [9], LayoutLM [24] and StructuralLM [15]. We also includes multilingual pre-trained models such as XLM-RoBERTa [3], LayoutXLM [25] and XYLayoutLM [7], as well as popular models that incorporate text, layout and image modalities like LayoutLMv2 [27], DocFormer [1], ERNIE-Layout [20] and LayoutLMv3 [10].

A.3 Graph Reordering Strategy

As Figure 5 shows, the raw order of document is an incomprehensible reading order. In graph reordering strategy, document structure is represented in the form of a hierarchical structure graph (the graph of yellow round nodes in Figure 5). Through the depth-first traversal method, we can sort the nodes between different levels. As the figure shown, we firstly traverse node 1 and its child nodes and then traverse node 4. For nodes of the same level, we use position relation to sort. For example, node 3 is at the top of node 5, so node 3 is sorted before node 5. With these two reordering rules, we achieve the reordering optimization on the raw order based on graph structure.

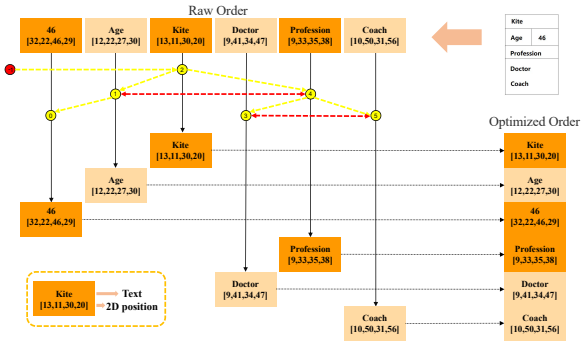


Figure 5: An example of Graph Reordering

A.4 Convergence Curves of the Ablation Study Fine-tuning

To increase the credibility of the experiment results, we also show the accuracy and F1 convergence curves of the ablation study fine-tuning. The line chart in the left of Figure 6 shows the convergence process of F1 on the FUNSD test set. We can see that the result with both graph reordering and graph masking methods is the best overall. Both of the results that incorporate the graph reorder and graph mask strategies respectively are overall better than the baseline. The line chart in the right of Figure 6 shows the convergence process of accuracy on XFUND. It can be seen together with Table 4 that only using the graph reordering strategy has little impact on the accuracy metric, while the graph masking strategy significantly improves accuracy.

Table 5: Ablation Study

Dataset	Graph Reorder	Graph Mask	Accuracy	Precision	Recall	F1
FUNSD	✗	✗	84.76	90.82	91.55	91.19
	✓	✗	85.70	92.36	93.15	92.75
	✗	✓	86.75	91.73	92.05	91.89
	✓	✓	88.39	92.46	93.85	93.15
CORD	✗	✗	97.11	96.35	96.71	96.53
	✓	✗	97.33	96.79	97.16	96.97
	✗	✓	97.88	96.94	97.23	97.09
	✓	✓	98.01	97.02	97.53	97.28
XFUND	✗	✗	85.87	89.79	93.94	91.82
	✓	✗	85.61	89.98	94.37	92.12
	✗	✓	90.88	91.58	94.43	92.99
	✓	✓	91.19	91.80	95.38	93.56

Table 6: Different Order.

Order	Accuracy	Precision	Recall	F1
Default	86.75	91.73	92.05	91.89
BFS	86.95	92.15	92.11	92.13
DFS	**88.39**	**92.46**	**93.85**	**93.15**

Table 7: Graph Masking Type.

Masking	Accuracy	Precision	Recall	F1
LayoutLMv3	84.76	90.82	91.55	91.19
GraphLayoutLM	**86.75**	**91.73**	**92.05**	**91.89**
w/o Graph Masking	82.96	90.32	91.54	90.88
w/ Hier Masking	86.09	90.40	91.80	91.09
w/ Sib Masking	85.60	91.77	90.85	91.31

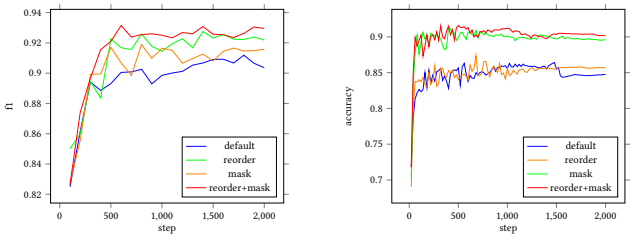


Figure 6: Convergence Process.

A.5 Results on other Datasets

We show the experiment results on SROIE and DocVQA in Table 4. Because we can not successfully reproduce layoutlmv3's results on these datasets, we only show them in the appendix. The experiment results show that our proposed method performs better than LayoutLMv3.

Table 4: Results on SROIE and DocVQA.

Methods	DocVQA(ANLS)	SROIE(F1)
LayoutLMv3-base	72.59	99.25
GraphLayoutLM-base(ours)	73.51	99.30

A.6 Ablation Study

We conduct ablation experiments on FUNSD, CORD and XFUND to test the effectiveness of our proposed graph reordering and graph masking strategies in GraphLayoutLM. We take LayoutLMv3 with a base size as the baseline model, add graph reordering and graph mask methods, respectively, and finally demonstrate the results of GraphLayoutLM combined with the two methods.

Table 5 shows the ablation experiment results on FUNSD, CORD and XFUND. It can be seen that the graph reordering method and graph mask method can improve precision, recall and F1. GraphLayoutLM, which combines the two strategies, performs better. It is worth noting that the graph mask strategy shows a more significant improvement in accuracy (1.99 on FUNSD, 0.77 on CORD and 5.01 on XFUND).

A.7 Further Analysis

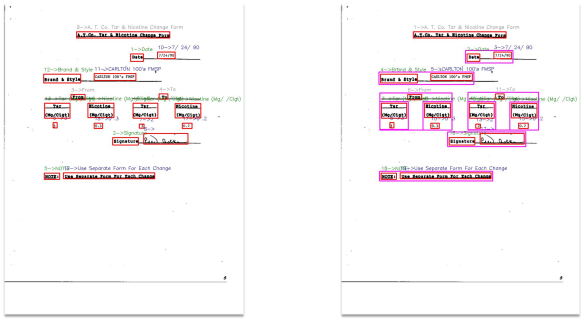
Graph Reordering Strategy. To further study the influence of different graph reordering algorithms, we conduct experiments on the FUNSD dataset using various algorithms. As shown in Table 6, "Default" represents the original order, "BFS" represents the order of breadth-first traversal generation and "DFS" represents the order of depth-first traversal generation. The results indicate that both "BFS" and "DFS" outperform "Default" in all evaluation indices. Specifically, "BFS" has an accuracy score and f1 score that are 0.20 and 0.24 higher than "Default", respectively, while "DFS" has an accuracy score and f1 score that are 1.64 and 1.26 higher than "Default", respectively. These findings suggest that a certain logical ordering is better than the original ordering. Additionally, "DFS" produces the best results among the algorithms tested, which verifies its superiority on our layout graph because the resulting order is more in line with human habits.

Graph Masking Strategy. We study the validity of different graph masking strategies and conduct additional experiments on the FUNSD dataset. The results are presented in Table 7. "w/o Graph Masking" indicates that only the self-attention layer is added to the model without any image mask. "w/ Hier Masking" and "w/ Sib Masking" indicate that only the hierarchical graph mask and sibling graph mask are added to the model, respectively. Our analysis result shows that "w/o Graph Masking" doesn't do better than LayoutLMv3 even though it has more parameters. This suggests that the key to performance improvement from our Graph Masking strategy is not due to the increase in the number of parameters. Adding graphs with incomplete relationship information, as done with "w/ Sib Masking," results in little gain for the model and may even have a negative impact on model optimization, while "w/ Hier Masking" leads to similar results. Our proposed GraphLayoutLM structure outperforms all other models in the table, indicating that the improvement in model performance is mainly due to our novel layout graph and graph utilization strategies.

A.8 The Graph Example on FUNSD

We show the importance of graph structure with an example from FUNSD. As shown in Figure 7, the red box in the figure is the box of the node and the number before the text above the box is the order. The purple box in document (b) is the domain scope of its first node. All other nodes in the purple box are children of this node. For example, "7/24/90" is a child of "Date". The position relationships are established at the same level. For example, a "left-right" relationship is established between "From" and "To" nodes.

As we can see, this graph structure represents the layout relationships in the document better. It's easy to see connections between nodes, such as "7/24/90" being the answer to "Date". Similarly, we can easily find that the order after Graph Reordering is more logical.



(a) Raw Order (a) Graph Order

Figure 7: An Graph Example on FUNSD

A.9 Graph Models Comparison

We compare our method with other graph structuring methods mentioned in related work. We also compare the results of our method with those of comparable models (ERNIE-mmLayout, ROPE, FormNet).

Liu et al. [16] use the GCN method and fully connect nodes in the graph, which can lead to significant time spent processing graph information. In contrast, GraphLayoutLM draws on the idea of GAT, which combines the graph structure with the attention mechanism based on GCN. Additionally, when modeling the document structure graph, we establish edge relationships based on level and position rather than full connection to strengthen node knowledge learning with edge relationships.

ERNIE-mmLayout [23] includes coarse-grained nodes (paragraph nodes) and fine-grained nodes (text nodes). We only use text nodes, which eliminates the need to deal with additional redundant data. Besides, for ERNIE-mmLayout and our method, the hierarchical connection manners between coarse-grained nodes and fine-grained nodes are similar and both of them adopt the tree mode. However, ERNIE-mmLayout establishes full connections between coarse-grained nodes and fine-grained nodes respectively, while our method only establishes connections between sibling nodes. As mentioned in our paper, we believe that this method containing prior knowledge can make the model pay more attention to the surrounding node relations of the same level nodes. More importantly, the graph of ERNIE-mmLayout has only two levels, while ours can be multi-level and more free (the depth of leaf nodes can be different). Finally, ERNIE-mmLayout uses a more complex network model while we only use a multi-head self-attention layer, which makes the model lighter and faster.

ROPE [14] sorting uses data to generate an order based on the GCN network and the method we proposed adopts the reordering rule with prior knowledge. ROPE's graph is based on the β-skeleton graph and focuses only on neighbor nodes, while our graph focuses not only on neighbor nodes but also on sibling nodes of the same level (even if they are far away).

FormNet [13] focuses on the surrounding information of nodes, and uses the Rich Attention part to analyze the attention of nodes and their surrounding nodes. The graph modeling we proposed not only focuses on the peripheral information of nodes (the location information of sibling nodes of leaf nodes), but also uses hierarchical relations to construct the location relations of distant nodes (such as the relationship between paragraphs). We believe that there may also be a strong connection between distant nodes at the same level (for example, the first and last paragraphs of an essay may have a strong coherence).

The experiment results are shown in Table 8, we can see that GraphLayoutLM have the best performance.

Table 8: F1 Results on FUNSD and CORD.

Methods	FUNSD	CORD
ROPE	57.22	-
ERNIE-mmLayout-base	86.02	97.23
GraphLayoutLM-base(ours)	**93.15**	**97.28**
FormNet	84.69	97.28
ERNIE-mmLayout-base	86.49	97.38
GraphLayoutLM-large(ours)	**94.39**	**94.39**